

Designing Verification Checkpoints to Mitigate Silent Errors in Agentic AI Pipelines

Abstract

Agentic artificial intelligence (AI) systems increasingly combine large language models (LLMs) with tools, retrieval systems, external environments, memory, planning modules, and execution loops. Unlike conventional single-turn language generation, these pipelines can propagate an incorrect intermediate state across multiple dependent actions, producing an apparently successful final result despite an earlier undetected failure. Such *silent errors* are particularly concerning because conventional end-to-end evaluation may identify that a task failed without revealing where or why the failure occurred. This paper examines verification checkpoints as an architectural mechanism for detecting, localizing, and containing such errors. Drawing on research on chain-of-thought reasoning, process supervision, verifiers, self-consistency, tool-using agents, self-correction, retrieval-augmented generation, and agent benchmarks, the paper develops a taxonomy of checkpoint types and proposes a layered verification architecture. The central argument is that verification should be treated not as a final-stage quality-control mechanism but as a distributed systems property: checkpoints should be placed at semantically meaningful boundaries where assumptions change, external state is consumed, irreversible actions are initiated, or uncertainty becomes consequential. The paper compares model-based, rule-based, execution-based, evidence-based, and human-in-the-loop verification. It further identifies limitations including correlated verifier errors, verification cost, reward hacking, observability gaps, specification ambiguity, and the difficulty of validating open-ended agent behavior. Finally, it proposes research directions involving adaptive checkpoint placement, heterogeneous verifier ensembles, provenance-aware verification, uncertainty-sensitive gating, causal error localization, and benchmark standards specifically designed to measure silent-error propagation.

Keywords: agentic AI, large language models, AI agents, verification, silent errors, process supervision, tool use, hallucination, reliability, checkpointing, self-correction, trustworthy AI

1. Introduction

Large language models have evolved from systems that primarily generate text into components of autonomous or semi-autonomous computational pipelines. Contemporary agents can reason over multi-step tasks, call external tools, retrieve information, maintain memory, interact with software environments, and iteratively revise their behavior. ReAct, for example, explicitly interleaves reasoning and action, allowing a language model to obtain information from an external environment while pursuing a task objective (Yao et al., 2023). Toolformer similarly demonstrates that language models can learn when to invoke APIs and how to incorporate returned information into subsequent generation (Schick et al., 2023).

This increased agency creates a reliability problem that differs qualitatively from ordinary hallucination. In a conventional question-answering system, an incorrect answer is generally the terminal failure. In an agentic pipeline, however, an incorrect intermediate decision may become an input to subsequent reasoning, retrieval, tool invocation, planning, or execution. The original error can therefore be transformed into a

sequence of internally coherent but externally incorrect actions. The resulting failure may be *silent*: the pipeline completes according to its control flow, yet the final state violates the user's objective or an environmental constraint.

This problem is amplified by the fact that successful execution is not equivalent to semantic correctness. A tool call may return a syntactically valid response that does not contain the requested information; a database query may execute successfully while selecting the wrong entity; a retrieval module may return documents that are relevant lexically but insufficient as evidence; or an agent may make a plausible inference from an incorrect premise. Recent research on multi-step tool-calling agents suggests that behavioral variation can occur even when the same task and system are repeatedly executed, illustrating that reliability encompasses both reasoning correctness and execution consistency.

Existing research provides several partial solutions. Chain-of-thought prompting exposes intermediate reasoning, self-consistency compares multiple reasoning trajectories, verifiers score candidate solutions, process supervision evaluates intermediate reasoning steps, and reflective agents use feedback to revise subsequent behavior (Wei et al., 2022; Wang et al., 2023; Cobbe et al., 2021; Lightman et al., 2024; Shinn et al., 2023). Yet these techniques do not by themselves establish a general architecture for *where* verification should occur in an agentic pipeline.

This paper therefore focuses on the design question: **How should verification checkpoints be positioned and implemented so that silent errors are detected before they propagate into consequential downstream states?** The paper treats verification as an architectural control mechanism rather than merely a prompting technique. It argues that effective verification requires a combination of checkpoint placement, appropriate verification modalities, evidence and provenance tracking, escalation policies, and explicit treatment of uncertainty.

2. Background and Related Work

2.1 Agentic AI pipelines

Agentic systems can be conceptualized as compositions of a language-model policy with perception, memory, planning, tool-use, and execution components. A simplified pipeline is:

[
I $\rightarrow$ P $\rightarrow$ R $\rightarrow$ T $\rightarrow$ O $\rightarrow$ U $\rightarrow$ E
]

where (I) denotes user intent, (P) planning, (R) reasoning, (T) tool selection and invocation, (O) observations, (U) updated internal state, and (E) external execution.

The ReAct framework established an influential pattern in which reasoning and acting are interleaved rather than separated into independent stages (Yao et al., 2023). Toolformer further demonstrated that tool invocation itself can be incorporated into language-model behavior (Schick et al., 2023).

Agent benchmarks reveal why reliability cannot be inferred from conventional language benchmarks. AgentBench evaluates LLMs across multiple interactive environments and identifies long-horizon reasoning, decision making, and instruction following as major obstacles (Liu et al., 2024). WebArena similarly evaluates agents in realistic web environments and reports a substantial gap between model and human end-to-end performance, emphasizing the difficulty of reliable autonomous interaction (Zhou et al., 2024).

2.2 Intermediate reasoning and process supervision

Chain-of-thought prompting showed that explicit intermediate reasoning can substantially improve performance on complex reasoning tasks (Wei et al., 2022). However, exposing a reasoning trace does not establish that the trace is correct.

A more direct approach is *process supervision*, in which intermediate steps receive correctness feedback rather than evaluating only the final answer. Lightman et al. (2024) compared process supervision with outcome supervision and found process supervision particularly effective for mathematical reasoning. Their PRM800K dataset contains 800,000 step-level correctness labels.

This distinction is central to agentic verification. If an agent makes an erroneous assumption at step (k), evaluating only the terminal state provides limited information about the source of failure. A checkpoint at step (k), by contrast, can prevent the incorrect state from becoming an input to steps (k+1, \ldots, n).

2.3 Verifiers and candidate selection

Verifier-based approaches separate generation from evaluation. Cobbe et al. (2021) trained verifiers to rank candidate mathematical solutions, demonstrating that verification can improve performance more effectively than simply increasing the generator's training data under their experimental conditions.

Self-consistency takes a related but weaker approach by sampling multiple reasoning paths and selecting the answer supported by the most consistent paths (Wang et al., 2023). The method produced substantial improvements on several reasoning benchmarks. However, agreement is not logically equivalent to correctness. Recent research explicitly demonstrates that LLMs can produce *self-consistent errors*: repeated generations may converge on the same incorrect answer, limiting the effectiveness of agreement-based verification (Tan et al., 2025).

2.4 Self-correction and reflection

Reflexion uses verbal feedback and episodic memory to allow an agent to modify subsequent behavior without updating model weights (Shinn et al., 2023). Such approaches demonstrate the value of feedback loops, but reflection is not necessarily independent verification. If the same model that generated an error is also responsible for judging it, correlated errors may remain undetected.

This distinction is important for checkpoint design: **self-critique is a verification mechanism only to the extent that the critic introduces sufficiently independent information, computation, evidence, or constraints.**

2.5 Factuality and retrieval verification

Factual verification research provides another foundation. SelfCheckGPT uses stochastic sampling to detect contradictions among generated passages, while RAGTruth demonstrates that retrieval augmentation does not eliminate unsupported or contradictory claims (Manakul et al., 2023; Niu et al., 2024).

FactScore approaches factual evaluation at the atomic-fact level, calculating the proportion of claims supported by reliable evidence (Min et al., 2023). This suggests a general principle for agents: verification should operate at the granularity at which errors can materially affect downstream decisions.

3. The Silent-Error Problem

A silent error is defined here as an erroneous intermediate or final state that does not trigger an explicit execution failure, exception, constraint violation, or human escalation.

Let an agentic pipeline contain states

[
 $S_0 \rightarrow S_1 \rightarrow \dots \rightarrow S_n$.
]

Suppose a transition produces an incorrect state (S_k^*), but the pipeline's control mechanism accepts it as valid. The probability that the final state is incorrect can be expressed conceptually as

[
 $P(F) = 1 - \prod_{i=1}^n (1 - p_i)$,
]

where (p_i) is the conditional probability that an undetected error at stage (i) produces a consequential failure. The equation is deliberately simplified: errors are usually correlated rather than independent. Nevertheless, it illustrates why a long pipeline can become substantially more fragile when intermediate states are unchecked.

Three properties distinguish silent errors from explicit failures:

1. **Syntactic validity:** the output conforms to the expected schema.
2. **Operational validity:** the tool or subsystem executes successfully.
3. **Semantic validity:** the result actually satisfies the intended constraint.

Most software infrastructure provides strong support for the first two properties. Agentic reliability requires substantially greater attention to the third.

For example, consider an agent asked to find a product meeting a price and specification constraint. A search tool may return a valid page; the agent may correctly parse the page; and the purchase API may

successfully execute. Yet the product could violate a critical specification. No infrastructure exception necessarily occurs. The system has therefore *succeeded operationally while failing semantically*.

4. Verification Checkpoints as an Architectural Primitive

4.1 Definition

A **verification checkpoint** is an explicit control boundary at which an agentic pipeline evaluates whether an intermediate state satisfies a predefined set of correctness, consistency, evidence, safety, or task-completion conditions before allowing downstream execution.

A checkpoint can be represented as

```
[  
Ci=(Si, Gi, Vi, Ai),  
]
```

where:

- (S_i) is the state being evaluated;
- (G_i) is the verification specification or invariant;
- (V_i) is the verifier;
- (A_i) is the action taken according to the verification result.

The action space should not be limited to *pass/fail*. A useful policy is

```
[  
Ai ∈ {\text{continue}, \text{repair}, \text{retry}, \text{branch}, \text{abstain}, \text{escalate}}.  
]
```

This transforms verification from a passive score into an active control mechanism.

4.2 Checkpoint placement

Checkpoints should not necessarily occur after every model generation. Excessive verification can increase latency and cost while creating additional opportunities for correlated model errors. Instead, checkpoint placement should follow *risk boundaries*.

High-value checkpoint locations include:

(a) Intent checkpoint.

Verify that the agent has correctly interpreted the user's objective, constraints, and success criteria.

(b) Evidence checkpoint.

Verify that retrieved information supports the claims being used for downstream reasoning.

(c) Planning checkpoint.

Verify that a proposed plan is complete, internally consistent, and compatible with environmental constraints.

(d) Tool-result checkpoint.

Verify that a tool response actually resolves the intended query rather than merely returning a technically valid response.

(e) State-transition checkpoint.

Verify that the new state follows logically from the previous state and observed evidence.

(f) Pre-action checkpoint.

Before an irreversible or consequential action, independently verify parameters, authorization, and expected effects.

(g) Completion checkpoint.

Evaluate whether the final state satisfies the original objective rather than merely whether the pipeline reached its terminal node.

5. Taxonomy of Verification Mechanisms

5.1 Rule-based verification

Deterministic rules are preferable where correctness can be specified formally. Examples include schema validation, numerical constraints, authorization policies, type checks, range constraints, and state-transition invariants.

Their principal advantage is independence from the model's linguistic judgment. Their limitation is expressiveness: many real-world objectives cannot be reduced to deterministic predicates.

5.2 Execution-based verification

When an artifact can be executed or tested, execution provides a strong external signal. Software engineering is an important example. SWE-bench evaluates whether generated patches resolve real GitHub issues through repository-level testing, illustrating the value of external execution as an evaluation mechanism (Jimenez et al., 2024).

Execution-based checks should be preferred over language-only evaluation whenever the task permits deterministic testing.

5.3 Evidence-based verification

Evidence-based checkpoints evaluate whether a claim or decision is supported by trustworthy external information. RAGTruth and FActScore illustrate the importance of explicit evidence when assessing factuality (Niu et al., 2024; Min et al., 2023).

An evidence checkpoint can require:

```
[
\text{Claim} \rightarrow \text{Evidence} \rightarrow \text{Entailment}.
]
```

The key design requirement is that retrieved evidence should not merely be present; it must actually entail or justify the proposition used by the agent.

5.4 Model-based verification

A second model can critique an output, score a candidate, or identify contradictions. Process reward models represent a particularly structured form of this approach (Lightman et al., 2024).

However, model-based verification introduces *correlated failure*. If generator and verifier share similar representations, training data, blind spots, or prompt-induced assumptions, both may produce the same error.

5.5 Cross-model and ensemble verification

Using multiple models or heterogeneous verification mechanisms can reduce—but not eliminate—correlated errors. Recent findings on self-consistent errors show why simple agreement should not be treated as proof of correctness (Tan et al., 2025).

An effective ensemble should therefore maximize *error diversity*, rather than merely model count. For example:

```
[
V = f(V_{\text{rules}}, V_{\text{execution}}, V_{\text{evidence}}, V_{\text{LLM}})
]
```

is potentially more robust than

```
[
V = f(V_{\text{LLM1}}, V_{\text{LLM2}}, V_{\text{LLM3}})
]
```

when the latter models share the same failure modes.

5.6 Human-in-the-loop verification

Human intervention is most valuable at high-impact uncertainty boundaries rather than as a universal final-stage review mechanism. A checkpoint can escalate when confidence falls below a threshold:

$$\left[\begin{array}{l} \text{Escalate if } P(\text{correct} \mid x) < \tau \\ \end{array} \right]$$

or when the action has unusually high consequences.

This creates a selective oversight architecture in which human attention is allocated according to risk.

6. A Layered Verification Architecture

A robust agentic pipeline can be organized into five layers.

Layer 1: Input and intent verification

The system extracts explicit goals, constraints, prohibited actions, and success criteria. Ambiguous requirements should trigger clarification rather than downstream inference.

Layer 2: State and evidence verification

Every major intermediate state should have provenance: what information produced it, which tools were used, when the information was retrieved, and which assumptions remain unresolved.

Layer 3: Process verification

Reasoning or planning steps are evaluated using deterministic constraints, process reward models, structured critics, or independent reasoning paths. The objective is early detection of the first consequential divergence.

Layer 4: Action verification

Before external side effects, the system verifies tool parameters, permissions, target identity, expected state changes, and reversibility.

Layer 5: Outcome verification

The system independently evaluates whether the final state satisfies the original objective. Importantly, this should not merely ask whether the agent *believes* it succeeded.

This architecture can be expressed as:

$$\begin{array}{l} I \\ \xrightarrow{C_1} \\ P \\ \xrightarrow{C_2} \end{array}$$

```

R
\rightarrow{C_3}
T
\rightarrow{C_4}
E
\rightarrow{C_5}
F.
]

```

A key design principle is **fail closed at consequential boundaries**: uncertainty should block or defer irreversible actions rather than silently propagate.

7. Current Approaches and Developments

Research has progressively moved from outcome-only evaluation toward richer process-aware approaches.

First, chain-of-thought methods established that intermediate reasoning can improve task performance (Wei et al., 2022). Second, self-consistency introduced multi-sample agreement as an inference-time reliability mechanism (Wang et al., 2023). Third, verifier-based methods explicitly separated generation from evaluation (Cobbe et al., 2021). Fourth, process supervision extended verification to intermediate steps, producing stronger signals than terminal outcome labels in some reasoning settings (Lightman et al., 2024). Fifth, agent frameworks increasingly incorporate reflection and iterative correction, as demonstrated by Reflexion (Shinn et al., 2023).

Tool-using agents create another important development. ReAct and Toolformer established mechanisms for integrating external tools into reasoning and decision making (Yao et al., 2023; Schick et al., 2023). This makes verification of tool observations particularly important: an agent can be wrong not because it failed to call a tool, but because it misunderstood what the tool actually returned.

Recent work also challenges the assumption that repeated agreement is sufficient evidence of correctness. Tan et al. (2025) find that self-consistent errors can persist across repeated generations and can remain difficult for conventional detection methods. Consequently, verification research is increasingly shifting from *consistency checking* toward *independent evidence and heterogeneous validation*.

8. Research Challenges and Limitations

8.1 Correlated verifier failures

The greatest limitation of LLM-based verification is that the verifier can inherit the generator's assumptions. A system may therefore create the appearance of redundancy without obtaining genuine independence.

8.2 Verification can itself hallucinate

A verifier may produce an incorrect critique, incorrectly reject a valid solution, or approve a flawed one. Thus, verification introduces another model-dependent failure surface.

8.3 Cost and latency

Suppose a pipeline contains (n) stages and each checkpoint adds cost (c_i) . Total verification overhead is

$$[\\ C_v = \sum_{i=1}^n c_i. \\]$$

For high-volume systems, indiscriminate checkpointing can become prohibitively expensive. Verification must therefore be risk-sensitive.

8.4 Specification ambiguity

Verification requires a criterion against which correctness can be judged. Many user goals are underspecified, subjective, or context-dependent. A checkpoint cannot reliably verify an objective that has not been operationalized.

8.5 Long-horizon credit assignment

If failure occurs after dozens of actions, identifying the first causal error may be difficult. AgentBench and WebArena demonstrate that long-horizon interaction introduces failure modes not captured by conventional short-form benchmarks.

8.6 Evaluation mismatch

Many benchmarks evaluate final task success without recording whether the system detected and recovered from intermediate errors. This makes it difficult to distinguish a robust pipeline from one that happens to reach a correct answer despite unsafe intermediate behavior.

8.7 Reward hacking and evaluator gaming

Once verification becomes part of the optimization target, agents may learn to satisfy the verifier rather than the underlying objective. This creates an evaluator-specification problem analogous to reward hacking in reinforcement learning.

8.8 Observability and provenance

Verification is difficult when intermediate state is not logged with sufficient detail. A system that records only final outputs cannot easily reconstruct which assumption, retrieval result, or tool call introduced an error.

9. Research Gaps and Future Directions

9.1 Adaptive checkpoint placement

Current research largely evaluates verification techniques rather than optimizing *where* verification should occur. Future systems should learn checkpoint placement as a risk-allocation problem.

One possible formulation is:

$$\begin{aligned} & [\\ & \max_{\mathcal{C}} \\ & \left[\right. \\ & R(\mathcal{C}) - \lambda C(\mathcal{C}) \\ & \left. \right] , \\ &] \end{aligned}$$

where $\mathcal{C}$ is the checkpoint set, R is expected reliability improvement, C is computational or human cost, and λ represents the cost sensitivity of the deployment.

9.2 Risk-sensitive verification

Not every action warrants the same level of scrutiny. A future checkpoint controller should consider:

- probability of error;
- severity of downstream consequences;
- reversibility of the action;
- uncertainty in the evidence;
- independence of available verifiers;
- cost of verification.

This would shift verification from a fixed pipeline property toward dynamic risk management.

9.3 Heterogeneous verification ensembles

Future systems should combine fundamentally different validation mechanisms rather than simply increasing the number of LLM judges. Deterministic rules, executable tests, retrieval evidence, symbolic checks, simulators, and independent models can provide complementary evidence.

9.4 Provenance-aware agent state

Each intermediate state should ideally carry a provenance graph:

$$\begin{aligned} & [\\ & S_i \rightarrow \{E_1, E_2, \dots, E_m\} \\ &] \end{aligned}$$

where (E_j) represents evidence, tool output, assumption, or prior state supporting (S_i) . Verification can then ask not only *whether a state looks plausible* but *why the system believes it*.

9.5 First-error localization

Research should move beyond detecting that an agent failed toward identifying the earliest consequential error. Process-supervision research demonstrates the value of step-level signals (Lightman et al., 2024), while automated process-supervision approaches such as OmegaPRM explore scalable identification of reasoning errors (Luo et al., 2024).

9.6 Benchmarks for silent-error propagation

A dedicated benchmark should measure at least four dimensions:

1. **Detection rate:** proportion of injected or naturally occurring errors detected.
2. **Localization accuracy:** ability to identify the first consequential error.
3. **Containment rate:** proportion of errors prevented from propagating.
4. **Recovery quality:** probability of reaching the correct final state after detection.

Such benchmarks would complement end-to-end success metrics such as those used in AgentBench, WebArena, and SWE-bench.

9.7 Independent verification as an information problem

A deeper theoretical direction is to characterize verifier independence. Two verifiers should not be considered independent merely because they are different models. Their error correlation should be empirically estimated:

$$\left[\begin{array}{l} \rho(V_i, V_j) = \\ \operatorname{corr} \\ \left(\begin{array}{l} \mathbf{1}[V_i \text{ wrong}], \\ \mathbf{1}[V_j \text{ wrong}] \end{array} \right) \end{array} \right]$$

An ensemble with low error correlation may be more valuable than a larger ensemble with highly correlated failures.

10. Conclusion

Agentic AI pipelines introduce a reliability problem that cannot be adequately addressed through final-output evaluation alone. Because agents operate through chains of reasoning, retrieval, tool use, state transitions, and external actions, a locally plausible but incorrect intermediate state can silently contaminate subsequent computation.

Verification checkpoints provide a promising architectural response. Their purpose should not be limited to judging final answers; rather, they should detect and contain errors at semantic boundaries where incorrect assumptions can propagate or produce consequential effects. The strongest architecture combines deterministic validation, execution-based testing, evidence checking, process supervision, heterogeneous model verification, and selective human escalation.

The literature demonstrates that intermediate supervision can improve reasoning reliability, verifiers can improve candidate selection, self-consistency can increase performance under some conditions, and reflection can enable iterative correction (Cobbe et al., 2021; Wang et al., 2023; Lightman et al., 2024; Shinn et al., 2023). At the same time, research on self-consistent errors demonstrates that agreement itself is not sufficient evidence of truth (Tan et al., 2025).

The central research challenge is therefore not simply to build *more verifiers*, but to build verification systems that are appropriately placed, sufficiently independent, evidence-grounded, uncertainty-aware, and capable of stopping error propagation. Future agentic systems should treat verification as a first-class systems primitive analogous to error handling, transaction validation, and access control in conventional software engineering.

References

1. Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., Kerr, J., Mueller, J., Ladish, J., Landau, J., Ndousse, K., Sellitto, M., Elhage, N., Schiefer, N., Mercado, N., DasSarma, N., Lasenby, R., Larson, R., Ringer, S., Johnston, S., Kravec, S., El Showk, S., Fort, S., Lanham, T., Telleen-Lawton, T., Conerly, T., Hume, T., Bowman, S. R., Hatfield-Dodds, Z., Mann, B., Amodei, D., Joseph, N., McCandlish, S., & Brown, T. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv:2212.08073.
2. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv: 2110.14168.
3. Cheng, F., Zouhar, V., Arora, S., Sachan, M., Strobel, H., & El-Assady, M. (2024). *RELIC: Investigating Large Language Model Responses using Self-Consistency*. Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, Article 647, 1–18. DOI: **10.1145/3613904.3641904**.
4. Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* International Conference on Learning Representations (ICLR 2024). arXiv:2310.06770.
5. Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2024). *Let's Verify Step by Step*. International Conference on Learning Representations (ICLR 2024). arXiv:2305.20050.

6. Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., & Tang, J. (2024). *AgentBench: Evaluating LLMs as Agents*. International Conference on Learning Representations (ICLR 2024). arXiv:2308.03688.
7. Luo, L., Liu, Y., Liu, R., Phatale, S., Guo, M., Lara, H., Shu, L., Zhu, Y., Meng, L., Sun, J., & Rastogi, A. (2024). *Improve Mathematical Reasoning in Language Models by Automated Process Supervision*. arXiv:2406.06592.
8. Manakul, P., Liusie, A., & Gales, M. J. F. (2023). *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models*. arXiv:2303.08896.
9. Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). *FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 12076–12096.
10. Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., & Zhang, T. (2024). *RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models*. arXiv: 2401.00396.
11. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv:2302.04761.
12. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2303.11366.
13. Tan, H., Sun, F., Liu, S., Su, D., Cao, Q., Chen, X., Wang, J., Cai, X., Wang, Y., Shen, H., & Cheng, X. (2025). *Too Consistent to Detect: A Study of Self-Consistent Errors in LLMs*. Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025), pp. 4755–4765. DOI: [10.18653/v1/2025.emnlp-main.238](https://doi.org/10.18653/v1/2025.emnlp-main.238).
14. Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., & Zhou, D. (2023). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. International Conference on Learning Representations (ICLR 2023). Persistent identifier: OpenReview paper ID [1PL1NIMMrw](https://openreview.net/forum?id=1PL1NIMMrw).
15. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems 35 (NeurIPS 2022), pp. 24824–24837. DOI: [10.52202/068431-1800](https://doi.org/10.52202/068431-1800); arXiv:2201.11903.
16. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR 2023). arXiv:2210.03629.

17. **Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Srivatsan, K., Xu, Z., Desai, A., Alon, U., Xu, Y., et al. (2024).** *WebArena: A Realistic Web Environment for Building Autonomous Agents*. International Conference on Learning Representations (ICLR 2024). The benchmark evaluates autonomous agents over 812 realistic web tasks and reports programmatic outcome-based evaluation.

Note on source integrity

The references above were selected from identifiable scholarly publications, conference proceedings, or arXiv records, and bibliographic identifiers have been included where they were available in the retrieved scholarly records. Recent 2025–2026 work is used selectively to capture developments directly relevant to self-consistent errors and agentic reliability; it should be distinguished from established peer-reviewed literature where appropriate. In particular, the 2025 EMNLP paper by Tan et al. is a peer-reviewed conference publication, whereas some newer 2026 agent-reliability results appearing in search results are preprints and are therefore not relied upon as foundational evidence.